

This Java 8 challenge tests your knowledge of [Lambda expressions](#)!

Write the following methods that *return a lambda expression* performing a specified action:

1. PerformOperation isOdd(): The lambda expression must return `true` if a number is odd or `false` if it is even.
2. PerformOperation isPrime(): The lambda expression must return `true` if a number is prime or `false` if it is composite.
3. PerformOperation isPalindrome(): The lambda expression must return `true` if a number is a palindrome or `false` if it is not.

Input Format

Input is handled for you by the locked stub code in your editor.

Output Format

The locked stub code in your editor will print `lines` of output.

Sample Input

The first line contains an integer, `n` (the number of test cases).

The subsequent lines each describe a test case in the form of space-separated integers: The first integer specifies the condition to check for (`1` for Odd/Even, `2` for Prime, or `3` for Palindrome). The second integer denotes the number to be checked.

```
5
1 4
2 5
3 898
1 3
2 12
```

Sample Output

```
EVEN
PRIME
PALINDROME
ODD
COMPOSITE
PROGRAM:
import java.io.*;
import java.util.*;
interface PerformOperation {
```

```

    boolean check(int a);
}
class MyMath {
    public static boolean checker(PerformOperation p, int num) {
        return p.check(num);
    }
    public PerformOperation isOdd() {
        return (a) -> a % 2 != 0;
    }
}

```

```

    public PerformOperation isPrime() {
        return (a) -> {
            if (a < 2) return false;
            if (a == 2) return true;
            if (a % 2 == 0) return false;

            for (int i = 3; i <= a / i; i += 2) {
                if (a % i == 0) return false;
            }

            return true;
        };
    }
}

```

```

    public PerformOperation isPalindrome() {
        return (a) -> {
            int original = a;
            int reversed = 0;

            while (a > 0) {
                reversed = reversed * 10 + a % 10;
            }
        };
    }
}

```

```

        a /= 10;
    }

    return original == reversed;
};
}

}

public class Solution {

    public static void main(String[] args) throws IOException {
        MyMath ob = new MyMath();
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int T = Integer.parseInt(br.readLine());
        PerformOperation op;
        boolean ret = false;
        String ans = null;
        while (T--> 0) {
            String s = br.readLine().trim();
            StringTokenizer st = new StringTokenizer(s);
            int ch = Integer.parseInt(st.nextToken());
            int num = Integer.parseInt(st.nextToken());
            if (ch == 1) {
                op = ob.isOdd();
                ret = ob.checker(op, num);
                ans = (ret) ? "ODD" : "EVEN";
            } else if (ch == 2) {
                op = ob.isPrime();
                ret = ob.checker(op, num);
                ans = (ret) ? "PRIME" : "COMPOSITE";
            } else if (ch == 3) {

```

```

    op = ob.isPalindrome();
    ret = ob.checker(op, num);
    ans = (ret) ? "PALINDROME" : "NOT PALINDROME";

}
System.out.println(ans);
}
}
}

```

You are given a list of student information: ID, FirstName, and CGPA. Your task is to rearrange them according to their CGPA in decreasing order. If two student have the same CGPA, then arrange them according to their first name in alphabetical order. If those two students also have the same first name, then order them according to their ID. No two students have the same ID.

Hint: You can use comparators to sort a list of objects. See the [oracle docs](#) to learn about comparators.

Input Format

The first line of input contains an integer , representing the total number of students. The next lines contains a list of student information in the following structure:

ID Name CGPA

Constraints

The name contains only lowercase English letters. The contains only integer numbers without leading zeros. The *CGPA* will contain, at most, 2 digits after the decimal point.

Output Format

After rearranging the students according to the above rules, print the first name of each student on a separate line.

Sample Input

```

5
33 Rumpa 3.68
85 Ashis 3.85

```

56 Samiha 3.75

19 Samara 3.75

22 Fahim 3.76

Sample Output

Ashis

Fahim

Samara

Samiha

Rumpa

PROGRAM:

```
import java.util.*;
```

```
class Student {
```

```
    private int id;
```

```
    private String fname;
```

```
    private double cgpa;
```

```
    public Student(int id, String fname, double cgpa) {
```

```
        this.id = id;
```

```
        this.fname = fname;
```

```
        this.cgpa = cgpa;
```

```
    }
```

```
    public int getId() {
```

```
        return id;
```

```
    }
```

```
    public String getFname() {
```

```
        return fname;
```

```
    }
```

```
public double getCgpa() {  
    return cgpa;  
}  
}
```

// Complete the code

```
public class Solution {  
    public static void main(String[] args) {  
        Scanner in = new Scanner(System.in);  
  
        int testCases = Integer.parseInt(in.nextLine());  
  
        List<Student> studentList = new ArrayList<Student>();  
  
        while (testCases > 0) {  
            int id = in.nextInt();  
            String fname = in.next();  
            double cgpa = in.nextDouble();  
  
            Student st = new Student(id, fname, cgpa);  
            studentList.add(st);  
  
            testCases--;  
        }  
  
        // Sort students  
        Collections.sort(studentList, new Comparator<Student>() {  
            @Override  
            public int compare(Student s1, Student s2) {
```

```

        // 1. CGPA in decreasing order
        int result = Double.compare(s2.getCgpa(), s1.getCgpa());

        if (result != 0) {
            return result;
        }

        // 2. First name in alphabetical order
        result = s1.getFname().compareTo(s2.getFname());

        if (result != 0) {
            return result;
        }

        // 3. ID in increasing order
        return Integer.compare(s1.getId(), s2.getId());
    }
});

// Print names
for (Student st : studentList) {
    System.out.println(st.getFname());
}

in.close();
}
}

import java.util.*;

```

```
class Student {  
    private int id;  
    private String fname;  
    private double cgpa;  
  
    public Student(int id, String fname, double cgpa) {  
        this.id = id;  
        this.fname = fname;  
        this.cgpa = cgpa;  
    }  
  
    public int getId() {  
        return id;  
    }  
  
    public String getFname() {  
        return fname;  
    }  
  
    public double getCgpa() {  
        return cgpa;  
    }  
}
```

// Complete the code

```
public class Solution {  
    public static void main(String[] args) {  
        Scanner in = new Scanner(System.in);  
  
        int testCases = Integer.parseInt(in.nextLine());
```



```
List<Student> studentList = new ArrayList<Student>();
```

```
while (testCases > 0) {
```

```
    int id = in.nextInt();
```

```
    String fname = in.next();
```

```
    double cgpa = in.nextDouble();
```

```
    Student st = new Student(id, fname, cgpa);
```

```
    studentList.add(st);
```

```
    testCases--;
```

```
}
```

```
// Sort students
```

```
Collections.sort(studentList, new Comparator<Student>() {
```

```
    @Override
```

```
    public int compare(Student s1, Student s2) {
```

```
        // 1. CGPA in decreasing order
```

```
        int result = Double.compare(s2.getCgpa(), s1.getCgpa());
```

```
        if (result != 0) {
```

```
            return result;
```

```
        }
```

```
        // 2. First name in alphabetical order
```

```
        result = s1.getFname().compareTo(s2.getFname());
```

```
        if (result != 0) {
```

```
            return result;
```

```
        }
```

```

        // 3. ID in increasing order
        return Integer.compare(s1.getId(), s2.getId());
    }
});

// Print names
for (Student st : studentList) {
    System.out.println(st.getFname());
}

in.close();
}
}

```

PROGRAM:

```

class Solution {
    public int[] sortByParity(int[] nums) {
        int index = 0;

        for (int i = 0; i < nums.length; i++) {
            if (nums[i] % 2 == 0) {
                int temp = nums[i];
                nums[i] = nums[index];
                nums[index] = temp;
                index++;
            }
        }

        return nums;
    }
}

```

```
}
```

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array. The *Player* class is provided in the editor below. It has two fields:

1. `name` : a string.
2. `score` : an integer.

Given an array of *Player* objects, write a comparator that sorts them in order of decreasing score. If or more players have the same score, sort those players alphabetically ascending by name. To do this, you must create a *Checker* class that implements the *Comparator* interface, then write an `int compare(Player a, Player b)` method implementing the [Comparator.compare\(T o1, T o2\)](#) method. In short, when sorting in ascending order, a comparator function returns `-1` if `a < b`, `0` if `a == b`, and `1` if `a > b`.

Declare a *Checker* class that implements the *comparator* method as described. It should sort first descending by score, then ascending by name. The code stub reads the input, creates a list of *Player* objects, uses your method to sort the data, and prints it out properly.

Example

Sort the list as . Sort first descending by score, then ascending by name.

Input Format

The first line contains an integer, `n`, the number of players.
Each of the next `n` lines contains a player's `name` and `score`, a string and an integer.

Constraints

- `1 ≤ n ≤ 100`
- Two or more players can have the same name.
- Player names consist of lowercase English alphabetic letters.

Output Format

You are not responsible for printing any output to stdout. Locked stub code in *Solution* will instantiate a *Checker* object, use it to sort the *Player* array, and print each sorted element.

Sample Input

```
5
amy 100
david 100
heraldo 50
aakansha 75
aleksa 150
```

Sample Output

aleksa 150

amy 100

david 100

aakansha 75

heraldo 50

Explanation

The players are first sorted descending by score, then ascending by name.

PROGRAM:

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array. The *Player* class is provided in the editor below. It has two fields:

1. *name* : a string.
2. *score* : an integer.

Given an array of *Player* objects, write a comparator that sorts them in order of decreasing score. If or more players have the same score, sort those players alphabetically ascending by name. To do this, you must create a *Checker* class that implements the *Comparator* interface, then write an *int compare(Player a, Player b)* method implementing the [Comparator.compare\(T o1, T o2\)](#) method. In short, when sorting in ascending order, a comparator function returns *if* , *if* , and *if* .

Declare a *Checker* class that implements the *comparator* method as described. It should sort first descending by score, then ascending by name. The code stub reads the input, creates a list of *Player* objects, uses your method to sort the data, and prints it out properly.

Example

Sort the list as . Sort first descending by score, then ascending by name.

Input Format

The first line contains an integer, *n*, the number of players.

Each of the next *n* lines contains a player's *name* and *score*, a string and an integer.

Constraints

- *n* is in the range $[1, 1000]$.
- Two or more players can have the same name.
- Player names consist of lowercase English alphabetic letters.

Output Format

You are not responsible for printing any output to stdout. Locked stub code in *Solution* will instantiate a *Checker* object, use it to sort the *Player* array, and print each sorted element.

Sample Input

```
5
amy 100
david 100
heraldo 50
aakansha 75
aleksa 150
```

Sample Output

```
aleksa 150
amy 100
david 100
aakansha 75
heraldo 50
```

Explanation

The players are first sorted descending by score, then ascending by name.

PROGRAM:

```
import java.util.*;

class Player {
    String name;
    int score;

    Player(String name, int score) {
        this.name = name;
        this.score = score;
    }
}

class Checker implements Comparator<Player> {

    @Override
    public int compare(Player a, Player b) {
```

```

        if (a.score != b.score) {
            return b.score - a.score; // descending score
        }

        return a.name.compareTo(b.name); // ascending name
    }
}

public class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();

        Player[] player = new Player[n];
        Checker checker = new Checker();

        for(int i = 0; i < n; i++){
            player[i] = new Player(scan.next(), scan.nextInt());
        }
        scan.close();

        Arrays.sort(player, checker);
        for(int i = 0; i < player.length; i++){
            System.out.printf("%s %s\n", player[i].name, player[i].score);
        }
    }
}

```

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array.

The *Player* class is provided for you in your editor. It has fields: a `String` and a `integer`.

Given an array of *Player* objects, write a comparator that sorts them in order of decreasing score; if or more players have the same score, sort those players alphabetically by name. To do this, you must create a *Checker* class that implements the *Comparator* interface, then write an *int compare(Player a, Player b)* method implementing the [Comparator.compare\(T o1, T o2\)](#) method.

Input Format

Input from stdin is handled by the locked stub code in the *Solution* class.

The first line contains an integer, *n*, denoting the number of players.
Each of the *n* subsequent lines contains a player's *name* and *score*, respectively.

Constraints

- *n* is between 1 and 100.
- players can have the same name.
- Player names consist of lowercase English letters.

Output Format

You are not responsible for printing any output to stdout. The locked stub code in *Solution* will create a *Checker* object, use it to sort the *Player* array, and print each sorted element.

Sample Input

```
5
amy 100
david 100
heraldo 50
aakansha 75
aleksa 150
```

Sample Output

```
aleksa 150
amy 100
david 100
aakansha 75
heraldo 50
```

PROGRAM:

```
import java.util.*;

class Checker implements Comparator<Player> {
    @Override
```

```

public int compare(Player a, Player b) {
    // Higher score first
    if (a.score != b.score) {
        return b.score - a.score;
    }

    // Same score: alphabetical name
    return a.name.compareTo(b.name);
}

class Player{
    String name;
    int score;

    Player(String name, int score){
        this.name = name;
        this.score = score;
    }
}

class Solution {

    public static void main(String[] args) {
        Scanner scan = new Scanner(System.in);
        int n = scan.nextInt();

        Player[] player = new Player[n];
        Checker checker = new Checker();

        for(int i = 0; i < n; i++){
            player[i] = new Player(scan.next(), scan.nextInt());

```



```

    }
    scan.close();

    Arrays.sort(player, checker);
    for(int i = 0; i < player.length; i++){
        System.out.printf("%s %s\n", player[i].name, player[i].score);
    }
}
}

```

1480. Running Sum of 1d Array

Solved

Easy

Topics

Companies

Hint

Given an array `nums`. We define a running sum of an array as `runningSum[i] = sum(nums[0]...nums[i])`.

Return the running sum of `nums`.

Example 1:

Input: `nums = [1,2,3,4]`

Output: `[1,3,6,10]`

Explanation: Running sum is obtained as follows: `[1, 1+2, 1+2+3, 1+2+3+4]`.

Example 2:

Input: `nums = [1,1,1,1,1]`

Output: `[1,2,3,4,5]`

Explanation: Running sum is obtained as follows: `[1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1]`.

Example 3:

Input: `nums = [3,1,2,10,1]`

Output: `[3,4,6,16,17]`

Constraints:

- $1 \leq \text{nums.length} \leq 1000$
- $-10^6 \leq \text{nums}[i] \leq 10^6$

PROGRAM:

```
class Solution {  
    public int[] runningSum(int[] nums) {  
        for (int i = 1; i < nums.length; i++) {  
            nums[i] = nums[i] + nums[i - 1];  
        }  
  
        return nums;  
    }  
}
```

1672. Richest Customer Wealth

Solved

Easy

Topics

Companies

Hint

You are given an $m \times n$ integer grid `accounts` where `accounts[i][j]` is the amount of money the i^{th} customer has in the j^{th} bank. Return *the **wealth** that the richest customer has*.

A customer's **wealth** is the amount of money they have in all their bank accounts. The richest customer is the customer that has the maximum **wealth**.

Example 1:

Input: `accounts = [[1,2,3],[3,2,1]]`

Output: 6

Explanation:

1st customer has $\text{wealth} = 1 + 2 + 3 = 6$

2nd customer has $\text{wealth} = 3 + 2 + 1 = 6$

Both customers are considered the richest with a wealth of 6 each, so return 6.

Example 2:

Input: accounts = [[1,5],[7,3],[3,5]]

Output: 10

Explanation:

1st customer has wealth = 6

2nd customer has wealth = 10

3rd customer has wealth = 8

The 2nd customer is the richest with a wealth of 10.

Example 3:

Input: accounts = [[2,8,7],[7,1,3],[1,9,5]]

Output: 17

Constraints:

- m == accounts.length
- n == accounts[i].length
- 1 <= m, n <= 50
- 1 <= accounts[i][j] <= 100

PROGRAM:

```
class Solution {  
    public int maximumWealth(int[][] accounts) {  
        int maxWealth = 0;  
  
        for (int i = 0; i < accounts.length; i++) {  
            int wealth = 0;  
  
            for (int j = 0; j < accounts[i].length; j++) {  
                wealth += accounts[i][j];  
            }  
  
            maxWealth = Math.max(maxWealth, wealth);  
        }  
    }  
}
```

```
        return maxWealth;
    }
}
```

977. Squares of a Sorted Array

Solved

Easy

Topics

Companies

Given an integer array `nums` sorted in **non-decreasing** order, return *an array of **the squares of each number** sorted in non-decreasing order.*

Example 1:

Input: `nums = [-4,-1,0,3,10]`

Output: `[0,1,9,16,100]`

Explanation: After squaring, the array becomes `[16,1,0,9,100]`.

After sorting, it becomes `[0,1,9,16,100]`.

Example 2:

Input: `nums = [-7,-3,2,3,11]`

Output: `[4,9,9,49,121]`

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-10^4 \leq \text{nums}[i] \leq 10^4$
- `nums` is sorted in **non-decreasing** order.

Follow up: Squaring each element and sorting the new array is very trivial, could you find an $O(n)$ solution using a different approach?

PROGRAM:

```
class Solution {
    public int[] sortedSquares(int[] nums) {
        int n = nums.length;
```

```

int[] result = new int[n];

int left = 0;
int right = n - 1;

for (int i = n - 1; i >= 0; i--) {
    if (Math.abs(nums[left]) > Math.abs(nums[right])) {
        result[i] = nums[left] * nums[left];
        left++;
    } else {
        result[i] = nums[right] * nums[right];
        right--;
    }
}

return result;
}

```

[724. Find Pivot Index](#)

Solved

Easy

Topics

Companies

Hint

Given an array of integers `nums`, calculate the **pivot index** of this array.

The **pivot index** is the index where the sum of all the numbers **strictly** to the left of the index is equal to the sum of all the numbers **strictly** to the index's right.

If the index is on the left edge of the array, then the left sum is 0 because there are no elements to the left. This also applies to the right edge of the array.

Return *the leftmost pivot index*. If no such index exists, return -1.

Example 1:

Input: nums = [1,7,3,6,5,6]

Output: 3

Explanation:

The pivot index is 3.

Left sum = nums[0] + nums[1] + nums[2] = 1 + 7 + 3 = 11

Right sum = nums[4] + nums[5] = 5 + 6 = 11

Example 2:

Input: nums = [1,2,3]

Output: -1

Explanation:

There is no index that satisfies the conditions in the problem statement.

Example 3:

Input: nums = [2,1,-1]

Output: 0

Explanation:

The pivot index is 0.

Left sum = 0 (no elements to the left of index 0)

Right sum = nums[1] + nums[2] = 1 + -1 = 0

Constraints:

- $1 \leq \text{nums.length} \leq 10^4$
- $-1000 \leq \text{nums}[i] \leq 1000$

PROGRAM:

```
class Solution {  
    public int pivotIndex(int[] nums) {  
        int total = 0;  
  
        for (int num : nums) {  
            total += num;  
        }  
    }  
}
```

```
int leftSum = 0;

for (int i = 0; i < nums.length; i++) {
    int rightSum = total - leftSum - nums[i];

    if (leftSum == rightSum) {
        return i;
    }

    leftSum += nums[i];
}

return -1;
}
```